

**PROYECTO BUSCAMINAS Y HUNDIR LA FLOTA
2ºDAM**

06/02/2026

iLERN.A.

ÍNDICE

PROYECTO BUSCAMINAS	3
¿Qué ha hecho cada integrante?	3
Rocío:	3
Carlos:	4
David:	6
PROYECTO HUNDIR LA FLOTA	7
¿Qué ha hecho cada integrante?	7
Fernando:	7
Lucía:	12
Alfonso:	13

PROYECTO BUSCAMINAS

Integrantes: Rocío Ramírez Sanromán, José Carlos López Gallego,
David José García García



¿Qué ha hecho cada integrante?

Rocío:

Función principal que maneja todo el juego

`def jugar(self):`

Inicia el juego con tablero 9x9 y 10 minas.

Muestra el tablero actual.

Pide jugada al usuario.

Procesa la jugada.

Verifica si ganó o perdió.
Repite hasta que termine.

Función que muestra el resultado final

`def mostrar_fin(self, gano):`

Si perdió (gano=False): muestra dónde estaban las minas.
Si ganó (gano=True): muestra felicitaciones.
Pregunta si quiere otra partida.

Carlos:

Función que inicializa el juego

`python def __init__(self, tamaño, numMinas):`

Crea un nuevo tablero con minas usando la clase Tablero.
Inicializa la matriz visible con "#" (casillas ocultas).
Inicializa la matriz marcada con False (sin banderas).
Establece el contador de casillas reveladas en 0.
Calcula el total de casillas seguras ($\text{tamaño}^2 - \text{minas}$).
Inicializa fin en False para controlar el estado del juego.

Función que convierte entrada del usuario a coordenadas

`python def parsear(self, entrada):`

Recibe entrada del usuario (ej: "1A", "5C").
Limpia espacios y convierte a mayúsculas.
Extrae el número como fila (resta 1 para índice base 0).
Extrae la letra como columna (convierte A→0, B→1, etc.).
Verifica que las coordenadas estén dentro del tablero.
Retorna (fila, columna) si es válida, (None, None) si no lo es.

Función que revela una casilla del tablero

`python def revelar(self, f, c):`

Verifica si la casilla ya está revelada.
Si es mina: muestra "*", termina el juego y retorna False.
Si no es mina: obtiene el valor (número de minas cercanas).
Muestra el valor en la matriz visible.
Incrementa el contador de casillas reveladas.

Si el valor es 0, llama a `_flood_fill()` para revelar automáticamente las adyacentes.

Retorna True si la jugada fue exitosa.

Función que revela casillas vacías automáticamente

`pythondef _flood_fill(self, f, c):`

Recorre las 8 casillas adyacentes a una casilla con valor 0.

Si una casilla adyacente está oculta, la revela.

Obtiene su valor y lo muestra.

Incrementa el contador de reveladas.

Si esa casilla también es 0, se llama recursivamente.

Expande el área revelada hasta encontrar números o bordes.

Función que marca/desmarca casillas con bandera

`pythondef marcar(self, f, c):`

Verifica que la casilla esté oculta (no revelada).

Si ya está revelada, muestra un mensaje de error.

Alterna el estado de marcado (True↔False).

Muestra "F" si se marcó o "X" si se desmarcó.

No afecta el estado del juego, solo es ayuda visual.

Función que verifica si el jugador ganó

`pythondef verificar_victoria(self):`

Compara casillas reveladas con total de casillas seguras.

Si son iguales, el jugador ha ganado.

Marca fin como True y retorna True.

Si no ha ganado, retorna False.

Se llama después de cada jugada exitosa.

Función que genera el tablero para mostrar al usuario

`pythondef tablero_mostrado(self):`

Crea una copia de la matriz visible.

Recorre todas las casillas del tablero.

Donde haya marcas de bandera (y casilla oculta), coloca "F".

Retorna la matriz combinada con casillas reveladas y banderas.

Se usa para mostrar el estado actual al jugador.

David:

Función que inicializa el tablero de juego

`pythondef __init__(self, tamaño, numMinas):`

Guarda el tamaño del tablero y número de minas.

Crea una matriz de ceros del tamaño indicado.

Coloca las minas aleatoriamente llamando a `_colocar_minas()`.

Calcula los números de minas cercanas llamando a `_calcular_minas()`.

Almacena la solución completa del juego.

Función que distribuye las minas en el tablero

`pythondef _colocar_minas(self):`

Genera posiciones aleatorias sin repetir usando `np.random.choice`.

Convierte las posiciones a coordenadas de fila y columna.

Marca con -1 las casillas que contienen minas.

Garantiza que no haya dos minas en la misma posición.

Función que calcula los números del buscaminas

`pythondef _calcular_minas(self):`

Recorre cada casilla del tablero.

Si es mina, coloca `"*"` en la solución.

Si no es mina, cuenta las minas en las 8 casillas adyacentes.

Guarda el conteo (0-8) en la solución.

Retorna la matriz completa con asteriscos y números.

Función que verifica si hay mina en una posición

`pythondef es_mina(self, f, c):`

Recibe coordenadas de fila y columna.

Comprueba si el valor en esa posición es -1.

Retorna True si es mina, False si no lo es.

Función que obtiene el valor de una casilla

`pythondef valor(self, f, c):`

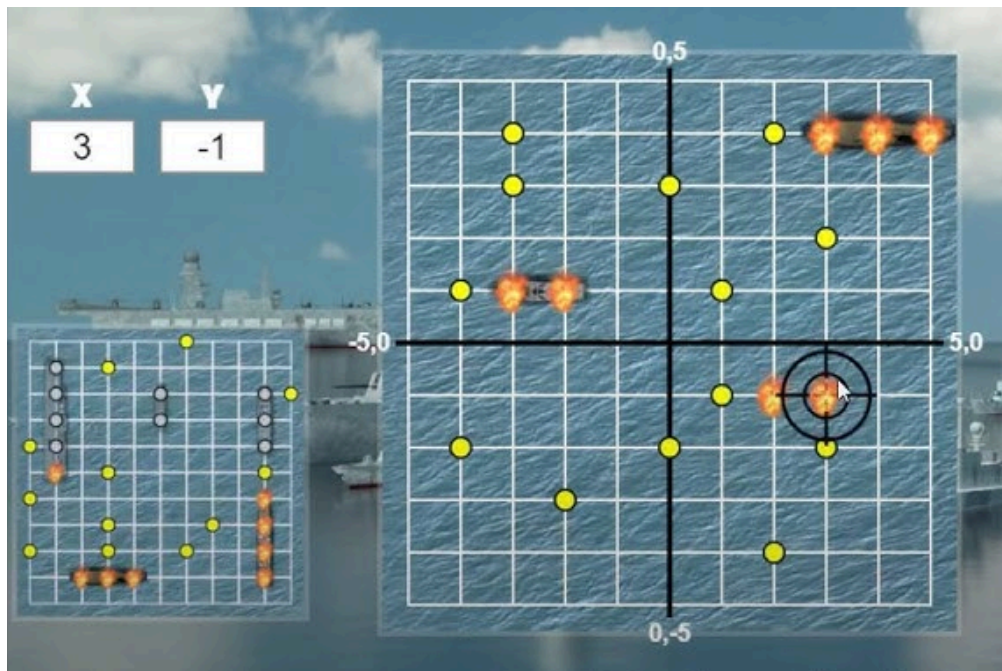
Recibe coordenadas de fila y columna.

Retorna el valor de la solución en esa posición.

Puede ser `"*"` (mina) o un número (0-8).

PROYECTO HUNDIR LA FLOTA

Integrantes: Fernando Ruiz Muriano, Lucía Jiménez Morales, Alfonso Rafael Mateo Navarro



¿Qué ha hecho cada integrante?

Fernando:

- Merging de ramas
- Normalización de variables entre ramas
- División en clases del código
- Modificación de funciones de otras ramas por decisiones de código o diseño
- Creación de dificultades

Clase User

Colocación automática de barcos (con ayuda de IA)

`def colocarBarcosAutomatico(barcos, tablero)`

Coloca los barcos del usuario de forma aleatoria en el tablero. Genera posiciones y orientaciones aleatorias, verifica que los barcos quepan y no se superpongan, y los marca con "1" en el tablero.

Solicitar coordenada de ataque

`def pensarAtaque(opcionesUser, controller):`

Pide al usuario una coordenada válida para atacar. Solicita entrada en formato "A10", valida que sea correcta y que no haya sido jugada antes.

Ejecutar ataque del usuario

`def atacar(coord, barcos, opciones, casillasTocadas):`

Ataca una coordenada del tablero enemigo. Elimina la coordenada de opciones disponibles, verifica si toca un barco y lo registra en casillas tocadas. Retorna True si impacta, False si es agua.

Clase GameController

Cabecera de turno (con ayuda de IA)

`def imprimirCabeceraTurno(turno, numero_turno):`

Imprime una cabecera visual indicando el turno actual (IA o USUARIO) y el número de turno con formato decorativo.

Imprimir tablero genérico (con ayuda de IA)

```
def imprimirTablero(tablero, ataques, impactos, mostrar_barcos, titulo):
```

Imprime un tablero con formato visual usando letras (A-J) para filas y números (1-10) para columnas. Muestra: "~" agua, "o" ataque fallido, "X" impacto, "■" barco (si mostrar_barcos=True).

Mostrar tablero de la IA al usuario (con ayuda de IA)

```
def imprimirTableroIAParaUsuario():
```

Muestra el tablero de la IA con solo los ataques realizados por el usuario, sin revelar la posición de los barcos enemigos.

Mostrar tablero del usuario (con ayuda de IA)

```
def imprimirTableroUsuario():
```

Muestra el tablero del usuario con sus barcos visibles y los ataques recibidos de la IA.

Crear opciones de movimiento

```
def crearOpciones(tamaño=10):
```

Genera un conjunto (set) con todas las coordenadas posibles del tablero (100 tuplas de 0,0 a 9,9) que representan las casillas disponibles para atacar.

Crear tablero vacío

```
def crearTablero(tamaño=10):
```

Crea un tablero vacío usando NumPy. Retorna un array bidimensional 10x10 lleno de ceros, donde 0=agua, 1=barco, -1=barco hundido.

Revisar estado de los barcos

`def revisarBarcos(barcos, casillasTocadas, tablero, casillasTocadasOponente):`

Verifica si algún barco ha sido hundido completamente comparando sus coordenadas con las casillas tocadas. Si un barco se hunde, limpia sus casillas tocadas, lo marca como hundido en el tablero (-1) y retorna la lista de barcos hundidos.

Traducir coordenadas

`def traducir(modo, coordenada):`

Convierte coordenadas entre formato visual (A-J, 1-10) y formato interno (0-9, 0-9). Modo "front" convierte interno→visual, modo "back" convierte visual→interno.

Inicializar partida

`def empezarJuego():`

Configura e inicia una nueva partida. Crea barcos y tableros para ambos jugadores, solicita dificultad (fácil/normal/difícil/rage bait), permite colocación automática o manual de barcos, elige aleatoriamente quién empieza y ejecuta el bucle de turnos.

Bucle principal de turnos

`def jugarTurnos(turno):`

Gestiona el flujo del juego alternando turnos entre IA y usuario. Ejecuta ataques, muestra resultados (agua/tocado/hundido), imprime tableros actualizados, permite turnos adicionales al tocar, y detecta la condición de victoria cuando todos los barcos de un jugador están hundidos.

Clase IA

Obtener casillas adyacentes

def obtenerAdyacentes(coordenada):

Calcula y retorna las coordenadas adyacentes válidas (arriba, abajo, izquierda, derecha) de una casilla dada, sin salirse del tablero 10x10.

Modo cacería (hunt)

def hunt(casillasTocadasIA, opcionesIA):

Activa el modo cacería cuando la IA ha tocado un barco. Si hay 2+ casillas tocadas alineadas, ataca los extremos de la línea. Si solo hay una tocada, ataca las casillas adyacentes. Filtra solo opciones válidas disponibles.

Decidir ataque de la IA

def pensarAtaque(casillasTocadasIA, barcosUser, tableroIA, opcionesIA, dificultad="difícil"):

Decide la coordenada de ataque según la dificultad: Fácil elige al azar, Rage bait conoce posiciones (50% del tiempo), Normal usa modo cacería al tocar, Difícil combina mapa de probabilidades + modo cacería eligiendo siempre la mejor opción.

Ejecutar ataque de la IA

def atacar(coord, barcosUser, opcionesIA, casillasTocadasIA):

Ataca una coordenada del tablero enemigo. Elimina la coordenada de opciones, verifica si toca un barco del usuario y lo registra en casillas tocadas. Retorna True si impacta, False si es agua.

Flujo del Programa

```
controller = GameController()  
user = User()  
ia = IA()  
controller.empezarJuego()
```

Se crean las instancias del controlador, usuario e IA, y se inicia la partida ejecutando `empezarJuego()`.

Lucía:

Creación del tablero del jugador con numpy

```
tablero = np.zeros((10, 10), dtype=int)
```

Se crea un tablero 10x10 inicializado a 0 - “0” casilla vacía, “1” casilla ocupada.

Creación de los barcos

```
def crearBarcos():
```

Devuelve una lista de barcos por su tamaño, la reutilizan tantos los jugadores como la IA.

Colocación manual de barcos

```
def colocarBarcos(barcos):
```

Permite al jugador colocar sus barcos manualmente.

Se muestra el tablero por pantalla y se ve: “~” para el agua, “■” para barcos.

Se pide al usuario que introduzca la fila, la columna y la orientación en la que quiere que vayan los barcos.

Se comprueba que la posición que quiere el usuario está dentro del tablero, que el barco quepa y que no se superpongan los barcos entre ellos.

Los barcos se marcarán con “1” en el tablero.

Patrón de ataque tipo ajedrez (IA)

`def aplicarPatronTablero(probabilidades, opcionesIA):`

Se aplica un patrón de tablero de ajedrez para poder mejorar los ataques iniciales de la IA.

Se incrementa la probabilidad de ataque en las casillas alternas ((fila + columna) % 2 == 0) para que se eviten ataques inútiles con los barcos grandes.

Tablero de la IA

`tablero_ia = np.zeros((10, 10), dtype=int)`

Se crea un tablero independiente para la IA con el mismo formato que el del jugador.

Colocación automática de barcos (IA)

`def colocarBarcosIA(barcos, tablero_ia):`

Coloca los barcos de la IA de forma automática y aleatoria.

Se elige una fila, columna y orientación aleatoria.

Se comprueba que el barco no se salga del tablero y no pise a otro barco, es decir, el mismo proceso que con la colocación manual, pero de forma aleatoria.

Se repite el proceso hasta que todos los barcos estén colocados.

Alfonso:

Verificación de la colocación de los barcos

`def puedeColocarBarco(tableroIA, fila, col, tamaño, orientacion, opcionesIA):`

Utilizando las distintas variables mencionadas arriba la función verifica la posición de los barcos para ver si se pueden colocar correctamente.

¿Entra el barco?

`def CabeBarco(tablero, fila, col, tamaño):`

Tomando el centro como punto de máximo valor calcula dónde puede entrar un barco enemigo.

Probabilidades

`def calcularProbabilidades(tableroIA, barcosRestantes, opcionesIA):`

Calcula la probabilidad de cada casilla según los barcos que caben.